

System execution context

Document #: P2079R3
Date: 2022-07-14
Project: Programming Language C++
Audience: SG1, LEWG
Reply-to: Lee Howes
<lwh@fb.com>
Ruslan Arutyunyan
<ruslan.arutyunyan@intel.com>
Michael Voss
<michaelj.voss@intel.com>

§ 1 Abtract

A standard execution context based on the facilities in [P2300] that implements parallel-forward-progress to maximise portability. A set of `system_contexts` share an underlying shared thread pool implementation, and may provide an interface to an OS-provided system thread pool.

§ 2 Changes

§ 2.1 R3

- Remove `execute_all` and `execute_chunk`. Replace with compile-time customization and a design discussion.
- Add design discussion about the approach we should take for customization and the extent to which the context should be implementation-defined.
- Add design discussion for an explicit `system_context` class.
- Add design discussion about priorities.

§ 2.2 R2

- Significant redesign to fit in P2300 model.
- Strictly limit to parallel progress without control over the level of parallelism.
- Remove direct support for task groups, delegating that to `async_scope`.

§ 2.3 R1

— Minor modifications

§ 2.4 R0

— first revision

§ 3 Introduction

[P2300] describes a rounded set of primitives for asynchronous and parallel execution that give a firm grounding for the future. However, the paper lacks a standard execution context and scheduler. It has been broadly accepted that we need some sort of standard scheduler.

As noted in [P2079R1], an earlier revision of this paper, the `static_thread_pool` included in later revisions of [P0443] had many shortcomings. This was removed from [P2300] based on that and other input.

This revision updates [P2079R1] to match the structure of [P2300]. It aims to provide a simple, flexible, standard execution context that should be used as the basis for examples but should also scale for practical use cases. It is a minimal design, with few constraints, and as such should be efficient to implement on top of something like a static thread pool, but also on top of system thread pools where fixing the number of threads diverges from efficient implementation goals.

Unlike in earlier versions of this paper, we do not provide support for waiting on groups of tasks, delegating that to the separate `async_scope` design in [P2519R0], because that is not functionality specific to a system context. Lifetime management in general should be considered delegated to `async_scope`.

The system context is of undefined size, supporting explicitly *parallel forward progress*. By requiring only parallel forward progress, any created parallel context is able to be a view onto the underlying shared global context. All instances of the `system_context` share the same underlying execution context. If the underlying context is a static thread pool, then all `system_contexts` should reference that same static thread pool. This is important to ensure that applications can rely on constructing `system_contexts` as necessary, without spawning an ever increasing number of threads. It also means that there is no isolation between `system_context` instances, which people should be aware of when they use this functionality. Note that if they rely strictly on parallel forward progress, this is not a problem, and is generally a safe way to develop applications.

The minimal extensions to basic parallel forward progress are to support fundamental functionality that is necessary to make parallel algorithms work:

- Cancellation: work submitted through the parallel context must be cancellable.
- Forward progress delegation: we must be able to implement a blocking operation that ensures forward progress of a complex parallel algorithm without special cases.

An implementation of `system_context` *should* allow link-time or compile-time replacement of the implementation such that the context may be replaced with an implementation that compiles and runs in a single-threaded process or that can be replaced with an appropriately configured system thread pool by an end-user. We do not attempt to specify here the mechanism by which this should be implemented.

Early feedback on the paper from Sean Parent suggested a need to allow the system context to carry no threads of its own, and take over the main thread. While in [P2079R2] we proposed `execute_chunk` and `execute_all`, these enforce a particular implementation on the underlying execution context. Instead we simplify the proposal by removing this functionality and assume that it is implemented by link-time or compile-time replacement of the context. We assume that the underlying mechanism to drive the context, should one be necessary, is implementation-defined. This allows custom hooks for an OS thread pool, or a simple `drive()` method in `main`.

§ 4 Design

§ 4.1 `system_context`

The `system_context` creates a view on some underlying execution context supporting *parallel forward progress*. A `system_context` must outlive any work launched on it.

```
class system_context {
public:
    system_context();
    ~system_context();

    system_context(const system_context&) = delete;
    system_context(system_context&&) = delete;
    system_context& operator=(const system_context&) = delete;
    system_context& operator=(system_context&&) = delete;

    implementation-defined-system_scheduler get_scheduler();
    size_t max_concurrency() const noexcept;
};
```

- On construction, the `system_context` may initialize a shared system context, if it has not been previously initialized.
- To support sharing of an underlying system context, two `system_context` objects do not guarantee task isolation. If work submitted by one can consume the thread pool, that can block progress of another.
- The `system_context` is non-copyable and non-moveable.
- The `system_context` must outlive work launched on it. If there is outstanding work at the point of destruction, `std::terminate` will be called.
- The `system_context` must outlive schedulers obtained from it. If there are outstanding schedulers at destruction time, this is undefined behavior.

- `get_scheduler` returns a `system_scheduler` instance that holds a reference to the `system_context`.
- `max_concurrency` will return a value representing the maximum number of threads the context may support. This is not a snapshot of the current number of threads, and may return `numeric_limits<size_t>::max`.

§ 4.2 `system_scheduler`

A `system_scheduler` is a copyable handle to a `system_context`. It is the means through which agents are launched on a `system_context`. The `system_scheduler` instance does not have to outlive work submitted to it. The `system_scheduler` is technically implementation-defined, but must be nameable. See later discussion for how this might work.

```
class implementation-defined-system_scheduler {
public:
    system_scheduler() = delete;
    ~system_scheduler();

    system_scheduler(const system_scheduler&);
    system_scheduler(system_scheduler&&);
    system_scheduler& operator=(const system_scheduler&);
    system_scheduler& operator=(system_scheduler&&);

    bool operator==(const system_scheduler&) const noexcept;

    friend implementation-defined-system_sender tag_invoke(
        std::execution::schedule_t, const implementation-defined-
        system_scheduler&) noexcept;
    friend std::execution::forward_progress_guarantee tag_invoke(
        std::execution::get_forward_progress_guarantee_t,
        const system_scheduler&) noexcept;
    friend implementation-defined-bulk_sender tag_invoke(
        std::execution::bulk_t,
        const system_scheduler&,
        Sh&& sh,
        F&& f) noexcept;
};
```

- `system_scheduler` is not independently constructable, and must be obtained from a `system_context`. It is both move and copy constructable and assignable.
- Two `system_schedulers` compare equal if they share the same underlying `system_context`.
- A `system_scheduler` has reference semantics with respect to its `system_context`. Calling any operation other than the destructor on a `system_scheduler` after the `system_context` it was created from is destroyed is undefined behavior, and that operation may access freed memory.
- The `system_scheduler`:

- satisfies the scheduler concept and implements the schedule customisation point to return an implementation-defined sender type.
- implements the `get_forward_progress_guarantee` query to return `parallel`.
- implements the bulk CPO to customise the bulk sender adapter such that:
 - When `execution::set_value(r, args...)` is called on the created receiver, an agent is created with parallel forward progress on the underlying `system_context` for each `i` of type `Shape` from 0 to `sh`, where `sh` is the shape parameter to the bulk call, that calls `f(i, args...)`.
- schedule calls on a `system_scheduler` are non-blocking operations.
- If the underlying `system_context` is unable to make progress on work created through `system_scheduler` instances, and the sender retrieved from scheduler is connected to a receiver that supports the `get_delegatee_scheduler` query, work may be scheduled on the scheduler returned by `get_delegatee_scheduler` at the time of the call to `start`, or at any later point before the work completes.

§ 4.3 system sender

```
class implementation-defined-system_sender {
public:
    friend pair<std::execution::system_scheduler, delegatee_scheduler>
        tag_invoke(
            std::execution::get_completion_scheduler_t<set_value_t>,
            const system_scheduler&) noexcept;
    friend pair<std::execution::system_scheduler, delegatee_scheduler>
        tag_invoke(
            std::execution::get_completion_scheduler_t<set_done_t>,
            const system_scheduler&) noexcept;

    template<receiver R>
        requires receiver_of<R>
    friend implementation-defined-operation_state
        tag_invoke(execution::connect_t, implementation-defined-
            system_sender&&, R&&);

    ...
};
```

`schedule` on a `system_scheduler` returns some implementation-defined sender type.

This sender satisfies the following properties:

- Implements the `get_completion_scheduler` query for the value and done channel where it returns a type that is logically a pair of an object that compares equal to itself, and a representation of delegatee scheduler that may be obtained from receivers connected with the sender.
- If connected with a receiver that supports the `get_stop_token` query, if that `stop_token` is stopped, operations on which `start` has been called, but are not yet

running (and are hence not yet guaranteed to make progress) **must** complete with `set_done` as soon as is practical.

- connecting the sender and calling `start()` on the resulting operation state are non-blocking operations.

§ 5 Design discussion and decisions

§ 5.1 To drive or not to drive

The earlier version of this paper, [P2079R2], included `execute_all` and `execute_chunk` operations to integrate with senders. In this version we have removed them because they imply certain requirements of forward progress delegation on the system context and it is not clear whether or not they should be called.

It is still an open question whether or not having such a standard operation makes the system context more or less portable. We can simplify this discussion to a single function:

```
void drive(system_context& ctx, sender auto snd);
```

Let's assume we have a single-threaded environment, and a means of customising the `system_context` for this environment. We know we need a way to donate `main`'s thread to this context, it is the only thread we have available. Assuming that we want a `drive` operation in some form, our choices are to:

- define our `drive` operation, so that it is standard, and we use it on this system.
- or allow the customisation to define a custom `drive` operation related to the specific single-threaded environment.

With a standard `drive` of this sort (or of the more complex design in [P2079R2]) we might write an example to use it directly:

```
system_context ctx;  
auto snd = on(ctx, doWork());  
drive(ctx, std::move(snd));
```

Without `drive`, we rely on an `async_scope` to spawn the work and some system-specific `drive` operation:

```
system_context ctx;  
async_scope scope;  
auto snd = on(ctx, doWork());  
scope.spawn(std::move(snd));  
custom_drive_operation(ctx);
```

The question is: what is more portable? It seems at first sight that a general `drive` function is more portable. Without it, how can we write a fully portable “hello world” example?

More broadly, it may not be more portable. First, we don't know whether or not we need to call it. Whether that drive call is needed is a function of whether the environment is single threaded or not. If it is not, say we have a normal Windows system with threads, we simply don't need to call it and the thread pool may not even have a way to process the donated main thread.

Further, we don't know the full set of single threaded environments. If this is a UI we may not want main to call the `system_context`'s drive, but rather that it will drive some UI event loop directly and `system_context` is simply a window to add tasks to that event loop. drive in this context is a confusing complication and might be harmful.

From the other angle, is an entirely custom drive operation, pulled in through whatever mechanism we have for swapping out the `system_context` portable? Most systems will not need such a function to be called. We do not in general need to on Windows, Linux, MacOS and similar systems with thread pool support. When we do need it, we have explicitly opted in to compiling or linking against a specific implementation of the `system_context` for the environment in question. On that basis, given the amount of other work we'd have to do to make the system work, like driving the UI loop, the small addition of also driving the `system_context` seems minor.

The authors recommendation here is that we allow drive to be unspecified in the standard, and to appear as a result of customisation of the system context where needed. However, this is a question we should answer.

§ 5.2 Making `system_context` implementation-defined

The system context aims to allow people to implement an application that is dependent only on parallel forward progress and to port it to a wide range of systems. As long as an application does not rely on concurrency, and restricts itself to only the system context, we should be able to scale from single threaded systems to highly parallel systems.

In the extreme, this might mean porting to an embedded system with a very specific idea of an execution context. Such a system might not have a multi-threading support at all, and thus the system context not only need run with single thread, but actually run on the system's only thread. We might build the context on top of a UI thread, or we might want to swap out the system-provided implementation with one from a vendor like Intel with experience writing optimised threading runtimes.

We need to allow customisation of the system context to cover this full range of cases. For a whole platform this is relatively simple. We assume that everything is an implementation-defined type. The `system_context` itself is a named type, but in practice is implementation-defined, in the same way that `std::vector` is implementation-defined at the platform level.

Other situations may offer a little less control. If we wish Intel to be able to replace the system thread pool with TBB, or Adobe to customise the runtime that they use for all of Photoshop to adapt to their needs, we need a different customisation mechanism.

To achieve this we see options:

- Link-time replaceability. This could be achieved using weak symbols, or by choosing a runtime library to pull in using build options.
- Compile-time replaceability. This could be achieved by importing different headers, by macro definitions on the command line or various other mechanisms.
- Run-time replaceability. This could be achieved by subclassing and requiring certain calls to be made early in the process.

Link-time replaceability is more predictable, in that it can be guaranteed to be application-global. The downside of link-time replaceability is that it requires defining the ABI and thus would require significant type erasure and inefficiency. Some of that inefficiency can be removed in practice with link-time optimisation.

Compile-time is simpler but would be easy to get wrong by mixing flags across the objects in the build. Both link-time and compile-time maybe difficult to describe in the standard.

Run-time is easy for us to describe in the standard, using interfaces and dynamic dispatch with well-defined mechanisms for setting the implementation. The downsides are that it is hard to ensure that the right context is set early enough in the process and that, like link-time replacement, it requires type erasure.

The other question is to what extent we need to specify this. We could simply say that implementations should allow customization and leave it up to QOI. We already know that full platform customisations are possible. This approach would delegate the decision of how to allow Intel to replace the platform context with TBB up to the platform implementor. It would rely on an agreement between the system vendor and the runtime vendor.

The authors do not have a recommendation, only a wish to see customisation available. We should decide how best to achieve it within the standard. Assuming we delegate customisation to the platform implementor, what wording would be appropriate for the specification, if any?

§ 5.3 Need for the `system_context` class

Our goal is to expose a global shared context to avoid oversubscription of threads in the system and to efficiently share a system thread pool. Underneath the `system_context` there is a singleton of some sort, potentially owned by the OS.

The question is how we expose the singleton. We have a few obvious options:

- Explicit context objects, as we've described in R2 and R3 of this paper, where a `system_context` is constructed as any other context might be, and refers to a singleton underneath.

- A global `get_system_context()` function that obtains a `system_context` object, or a reference to one, representing the singleton explicitly.
- A global `get_system_scheduler()` function that obtains a scheduler from some singleton system context, but does not explicitly expose the context.

The `get_system_context()` function returning by value adds little, it's equivalent to direct construction. `get_system_context()` returning by reference and `get_system_scheduler()` have a different lifetime semantic from directly constructed `system_context`.

The main reason for having an explicit by-value context is that we can reason about lifetimes. If we only have schedulers, from `get_system_context().get_scheduler()` or from `get_system_scheduler()` then we have to think about how they affect the context lifetime. We might want to reference count the context, to ensure it outlives the schedulers, but this adds cost to each scheduler use, and to any downstream sender produced from the scheduler as well that is logically dependent on the scheduler. We could alternatively not reference count and assume the context outlives everything in the system, but that leads quickly to shutdown order questions and potential surprises.

By making the context explicit we require users to drain their work before they drain the context. In debug builds, at least, we can also add reference counting so that destruction of the context before work completes reports a clear error to ensure that people clean up. That is harder to do if the context is destroyed at some point after main completes. This lifetime question also applies to construction: we can lazily construct a thread pool before we first use a scheduler to it.

For this reason, and consistency with other discussions about structured concurrency, we opt for an explicit context object here.

§ 5.4 Priorities

It's broadly accepted that we need some form of priorities to tweak the behaviour of the system context. This paper does not include priorities, though early drafts of R2 did. We had different designs in flight for how to achieve priorities and decided they could be added later in either approach.

The first approach is to expand one or more of the APIs. The obvious way to do this would be to add a priority-taking version of `system_context::get_scheduler()`:

```
implementation-defined-system_scheduler get_scheduler();
implementation-defined-system_scheduler get_scheduler(priority_t
    priority);
```

This approach would offer priorities at scheduler granularity and apply to large sections of a program at once.

The other approach, which matches the receiver query approach taken elsewhere in [P2300] is to add a `get_priority()` query on the receiver, which, if available, passes a priority to the scheduler in the same way that we pass an allocator or a `stop_token`. This would work at task granularity, for each `schedule()` call that we connect a receiver to we might pass a different priority.

In either case we can add the priority in a separate paper. It is thus not urgent that we answer this question, but we include the discussion point to explain why they were removed from the paper.

§ 6 Examples

As a simple parallel scheduler we can use it locally, and `sync_wait` on the work to make sure that it is complete. With forward progress delegation this would also allow the scheduler to delegate work to the blocked thread. This example is derived from the Hello World example in [P2300]. Note that it only adds a well-defined context object, and queries that for the scheduler. Everything else is unchanged about the example.

```
using namespace std::execution;

system_context ctx;
scheduler auto sch = ctx.scheduler();

sender auto begin = schedule(sch);
sender auto hi = then(begin, []{
    std::cout << "Hello world! Have an int.";
    return 13;
});
sender auto add_42 = then(hi, [](int arg) { return arg + 42; });

auto [i] = this_thread::sync_wait(add_42).value();
```

We can structure the same thing using `execution::on`, which better matches structured concurrency:

```
using namespace std::execution;

system_context ctx;
scheduler auto sch = ctx.scheduler();

sender auto hi = then(just(), []{
    std::cout << "Hello world! Have an int.";
    return 13;
});
sender auto add_42 = then(hi, [](int arg) { return arg + 42; });

auto [i] = this_thread::sync_wait(on(sch, add_42)).value();
```

The `system_scheduler` customises bulk, so we can use bulk dependent on the scheduler. Here we use it in structured form using the parameterless `get_scheduler` that retrieves the

scheduler from the receiver, combined with on:

```
auto bar() {
    return
        ex::let_value(
            ex::get_scheduler(),           // Fetch scheduler from receiver.
            [](auto current_sched) {
                return bulk(
                    current_sched.schedule(),
                    1,                     // Only 1 bulk task as a lazy way of
                    making cout safe
                    [](auto idx){
                        std::cout << "Index: " << idx << "\n";
                    })
            });
}

void foo()
{
    using namespace std::execution;

    system_context ctx;

    auto [i] = this_thread::sync_wait(
        on(
            ctx.scheduler(),               // Start bar on the system_scheduler
            bar()))                       // and propagate it through the
        receivers
        .value();
}
```

Use `async_scope` and a custom system context implementation linked in to the process (through a mechanism undefined in the example). This might be how a given platform exposes a custom context. In this case we assume it has no threads of its own and has to take over the main thread through an custom `drive()` operation that can be looped until a callback requests exit on the context.

```
using namespace std::execution;

system_context ctx;

int result = 0;

{
    async_scope scope;
    scheduler auto sch = ctx.scheduler();

    sender auto work =
        then(just(), [&](auto sched) {

            int val = 13;

            auto print_sender = then(just(), [val]{
                std::cout << "Hello world! Have an int with value: " << val <<
                "\n";
            });
        });
}
```

```

});

// spawn the print sender on sched to make sure it
// completes before shutdown
scope.spawn(on(sch, std::move(print_sender)));

return val;
});

scope.spawn(on(sch, std::move(work)));

// This is custom code for a single-threaded context that we have
// replaced
// at compile-time (see discussion options).
// We need to drive it in main.
// It is not directly sender-aware, like any pre-existing work loop, but
// does provide an exit operation. We may call this from a callback
// chained
// after the scope becomes empty.
// We use a temporary terminal_scope here to separate the shut down
// operation and block for it at the end of main, knowing it will
// complete.
async_scope terminal_scope;
terminal_scope.spawn(
    scope.on_empty() | then([](my_os::exit(ctx))));
my_os::drive(ctx);
this_thread::sync_wait(terminal_scope);
};

// The scope ensured that all work is safely joined, so result contains 13
std::cout << "Result: " << result << "\n";

// and destruction of the context is now safe

```

§ 7 References

[P0443] 2020. A Unified Executors Proposal for C++.

<https://wg21.link/p0443>

[P2079R1] Ruslan Arutyunyan, Michael Voss. 2020-08-15. Parallel Executor.

<https://wg21.link/p2079r1>

[P2079R2] Lee Howes, Ruslan Arutyunyan, Michael Voss. 2022-01-15. System execution context.

<https://wg21.link/p2079r2>

[P2300] 2022. std::execution.

<https://wg21.link/p2300>

[P2519R0] 2022. async_scope - Creating scopes for non-sequential concurrency.

<https://wg21.link/p2519>